

Projekt	Usługi sieciowe w biznesie	 POLITECHNIKA RZESZOWSKA im. IGNACEGO ŁUKASIEWICZA
Temat	System obsługi e parafii	
Kierunek / Grupa	Inżynieria i analiza danych	L2
Data	09.06.2026	
Repozytorium	https://github.com/Slasq/eParafia	

Organizacja projektu

1. Grupa projektowa jest zespołem deweloperskim pracującym zwinnie.
2. W zespole są reprezentowane poszczególne dyscypliny wytwórcze: analityk, projektant, programista, tester.
3. Produktem pracy zespołu jest działające i udokumentowane oprogramowanie.
4. Produkty analityka: wstępny opis wymagań, dokumentacja odkrywania encji, analityczny model dziedziny w postaci diagramu UML, wyjaśnienie modelu, dokumentacja odkrywania przypadków użycia, katalog przypadków użycia w postaci diagramu UML, opis przypadków użycia.
5. Produkty projektanta: model projektowy w stylu Domain-Driven Design zaprezentowany za pomocą diagramów UML, relacyjny model danych (UML lub ERD), inne diagramy UML (np. diagram sekwencji, diagram stanów).
6. Produkty programisty: kod programu Java/Spring/JPA/REST, skrypty DDL tworzące bazę danych.
7. Produkty testera: skrypty DML wypełniające bazę danymi testowymi, kolekcja testów (Bruno lub inne narzędzie), raport z testów.
8. Rozmiar modelu dziedziny: 20 encji.
9. Liczba opisanych i zaimplementowanych przypadków użycia: 5.
10. Dziedzinę (biznes) ustala zespół.
11. Wszystkie artefakty są składowane w repozytorium Git.
12. Data prezentacji: 16/17.06.2026.

Spis treści

1	Wprowadzenie.....	3
2	Analiza	3
2.1	Opis wymagań	3
2.2	Model dziedziny	4
2.2.1	Odkrywanie pojęć	4
2.2.2	Szkic modelu dziedziny	6
2.2.3	Wyjaśnienie modelu.....	12
2.3	Model przypadków użycia.....	14
2.3.1	Odkrywanie przypadków użycia i aktorów.....	14
2.3.2	Diagram przypadków użycia.....	14
2.3.3	Opis przypadków użycia	17
3	Projekt.....	22
4	Implementacja	22
4.1	Architektura aplikacji	22
4.2	Konteksty domenowe.....	22
4.3	Encje JPA i persystencja danych.....	22
4.4	Agregaty domenowe.....	23
4.5	Interfejs REST API.....	23
4.6	Obsługa błędów	24
4.7	Inicjalizacja i konfiguracja.....	24
4.8	Dokumentacja interaktywna API	24
5	Testy	26
5.1	Cel testów	26
5.2	Organizacja testów	26
5.3	Przygotowanie danych testowych	28
5.4	Testy opieki duszpasterskiej i kartotek (Pastoral Care)	28
5.5	Testy grup parafialnych (Parish Groups).....	30
5.6	Testy organizacji wydarzeń i harmonogramów (Event Coordination)	31
5.7	Podsumowanie	32
6	Informacje dodatkowe	32
6.1	Role w projekcie.....	32

1 Wprowadzenie

System eParafia wspiera kompleksowe zarządzanie działalnością parafii. Jego głównym celem jest usprawnienie codziennej pracy administracyjnej oraz duszpasterskiej poprzez uporządkowanie i cyfryzację kluczowych procesów związanych z funkcjonowaniem wspólnoty parafialnej. Dzięki integracji wielu obszarów działalności w jednym systemie możliwe jest efektywne zarządzanie zarówno danymi wiernych, jak i wydarzeniami religijnymi, dokumentacją czy zasobami personalnymi.

2 Analiza

2.1 Opis wymagań

System eParafia umożliwia kompleksowe zarządzanie działalnością **parafii**, obejmując obsługę **wiernych, sakramentów, intencji mszalnych, wydarzeń** oraz administracji parafialnej.

System zawiera podstawowe informacje o **parafii** takie, jak jej nazwa, adres, telefon, email, datę erygowania oraz **miejscowość i diecezję**, do jakiej należy.

Podstawową funkcjonalnością systemu jest prowadzenie **kartotek parafian** oraz rejestracja **zdarzeń religijnych** związanych z ich życiem duchowym. Każdy **parafianin** posiada swoją kartotekę zawierającą dane osobowe, adresowe oraz kontaktowe.

System umożliwia rejestrowanie **sakramentów** udzielanych parafianom, takich jak chrzest, bierzmowanie, małżeństwo. Każdy sakrament zawiera informacje o dacie, miejscu, osobie udzielającej oraz powiązanych uczestnikach.

W systemie prowadzony jest **harmonogram** wydarzeń religijnych np. mszy świętych wraz z przypisanymi **intencjami**. Intencje mogą być zamawiane przez parafian i zawierają treść, datę realizacji oraz status realizacji.

Dodatkowo system umożliwia zarządzanie wydarzeniami parafialnymi, takimi jak rekolekcje, spotkania grup czy uroczystości religijne. Wydarzenia mogą mieć przypisanych **uczestników** oraz **organizatorów**.

System umożliwia zarządzanie **grupami parafialnymi** (np. ministranci, schola), do których przypisywani są parafianie.

W systemie każdy parafianin przypisany jest do konkretnej **rodziny**, a każda rodzina do konkretnego **adresu**.

System wspiera także zarządzanie personelem parafii (**księża, pracownicy świeccy**), ich rolami, **stanowiskami** oraz **obowiązkami**.

W ramach funkcjonalności administracyjnych system umożliwia prowadzenie ewidencji **ofiar** (datków), zarządzanie **ogłoszeniami parafialnymi** oraz **dokumentacją**.

2.2 Model dziedziny

2.2.1 Odkrywanie pojęć

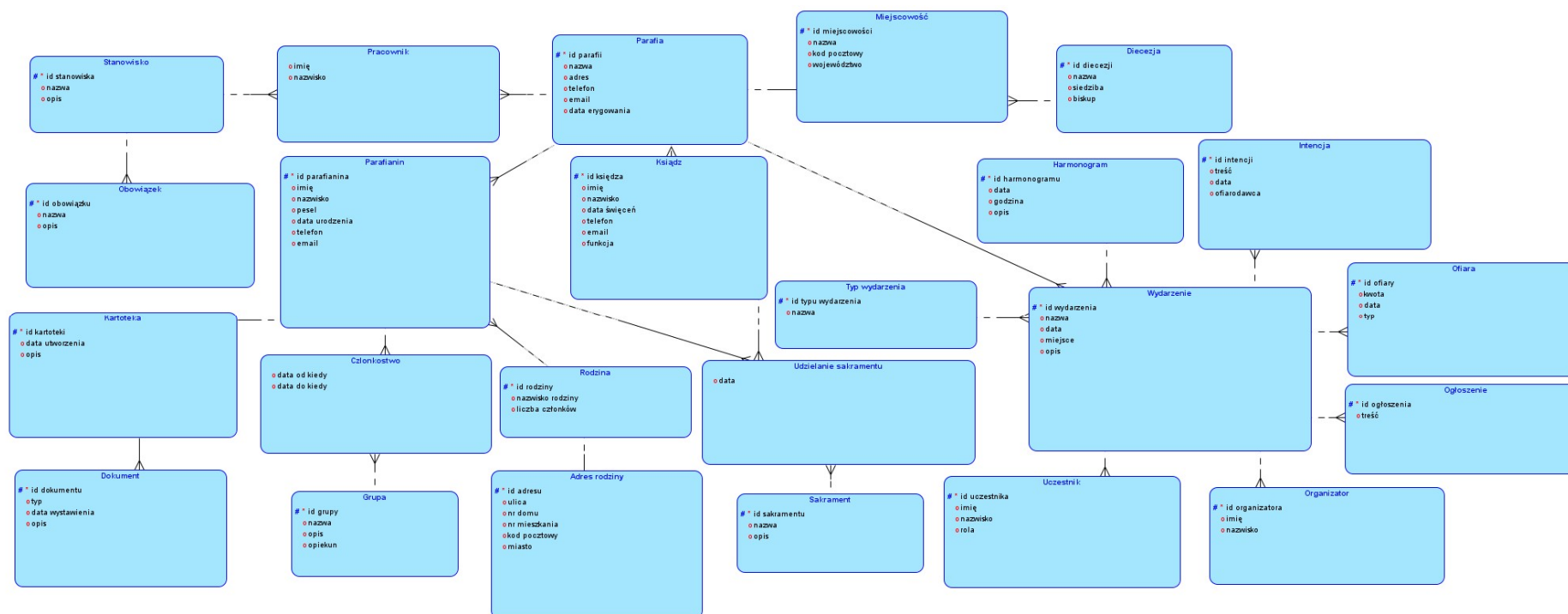
System obsługi e parafii składa się z następujących encji:

Encja	Opis
Parafia	Parafia zawiera podstawowe informacje identyfikujące jednostkę, takie jak id_parafii, nazwa, adres, telefon, email oraz data erygowania.
Miejscowość	Miejscowość określa miejsce, gdzie znajduje się parafia i zawiera informacje o nazwie, kodzie pocztowym oraz województwie, a także odniesienie do diecezji.
Diecezja	Diecezja zawiera informacje identyfikujące jednostkę, takie jak id diecezji, nazwa, siedziba i biskup.
Pracownik	Pracownik zawiera imię i nazwisko pracownika zatrudnionego na konkretnym stanowisku oraz powiązania ze stanowiskiem i parafią.
Stanowisko	Stanowisko określa id, nazwę oraz opis funkcji.
Obowiązek	Obowiązek opisuje konkretne zadania przypisane do stanowiska poprzez nazwę i opis.
Parafianin	Parafianin przechowuje dane wiernych, takie jak id_parafianina, imię, nazwisko, PESEL, data urodzenia, telefon i email, a także powiązania z parafią i rodziną.
Kartoteka	Kartoteka zawiera id_kartoteki, datę utworzenia oraz opis.
Dokument	Dokument posiada id_dokumentu, typ, datę wystawienia, opis oraz powiązanie z kartoteką.
Rodzina	Rodzina zawiera id_rodziny, nazwisko rodziny oraz liczbę członków. Każdy parafianin należy do konkretnej rodziny.
Adres_rodziny	Rodzina posiada przypisany adres_rodziny , gdzie zapisywane są szczegóły lokalizacji, takie jak ulica, numer domu i mieszkania, kod pocztowy oraz miasto.
Członkostwo	Członkostwo określa okres przynależności (data_od_kiedy, data_do_kiedy) oraz powiązania z grupą i parafianinem. Poprzez te encję parafianie mogą należeć do różnych grup
Grupa	Grupa posiada id_grupy, nazwę, opis oraz opiekuna.
Ksiądz	Ksiądz posiada dane osobowe takie, jak imię, nazwisko, telefon, email, datę święceń, funkcji (proboszcz, wikariusz) oraz dane parafii, która zarządza.
Udzielanie_sakramentu	Udzielanie_sakramentu łączy parafianina, księdza i sakrament, przechowując informacje o dacie udzielenia.
Sakrament	Sakrament zawiera id, nazwę i opis.
Wydarzenie	Wydarzenie przechowuje id, nazwę, datę, miejsce i opis oraz powiązania z parafią, typem wydarzenia i harmonogramem.
Typ_wydarzenia	Typ_wydarzenia definiuje kategorię wydarzenia poprzez id i nazwę.
Harmonogram	Harmonogram zawiera datę, godzinę oraz opis, porządkując przebieg wydarzeń.
Intencja	Intencja zawiera id_intencji, treść, datę oraz ofiarodawcę oraz powiązanie z konkretnym wydarzeniem.

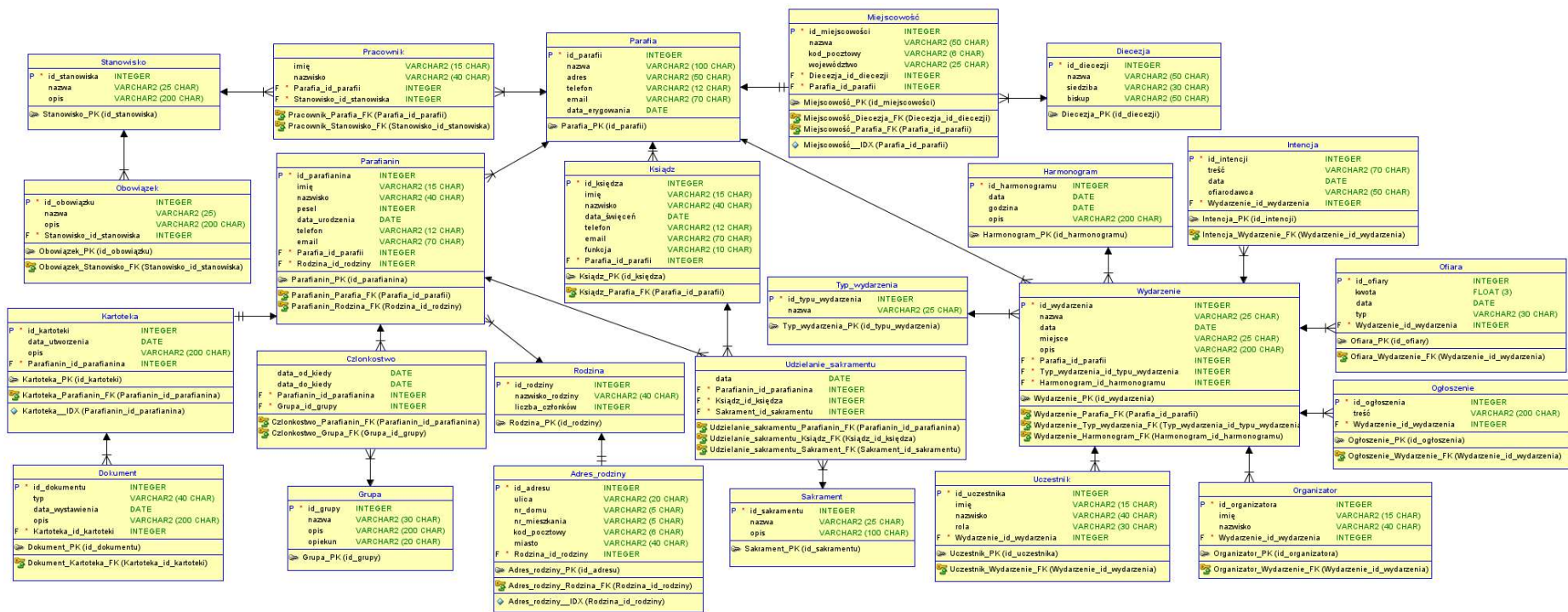
Ofiara	Ofiara zawiera kwotę, datę oraz typ datku oraz powiązanie z konkretnym wydarzeniem.
Ogłoszenie	Ogłoszenie zawiera id oraz treść oraz powiązanie z konkretnym wydarzeniem.
Organizator	Organizator przechowuje dane osób (imię, nazwisko, rola) oraz powiązanie z konkretnym wydarzeniem.
Uczestnik	Uczestnik przechowuje dane osób (imię, nazwisko, rola) oraz powiązanie z konkretnym wydarzeniem.

Tabela 1. Opis encji

2.2.2 Szkic modelu dziedziny

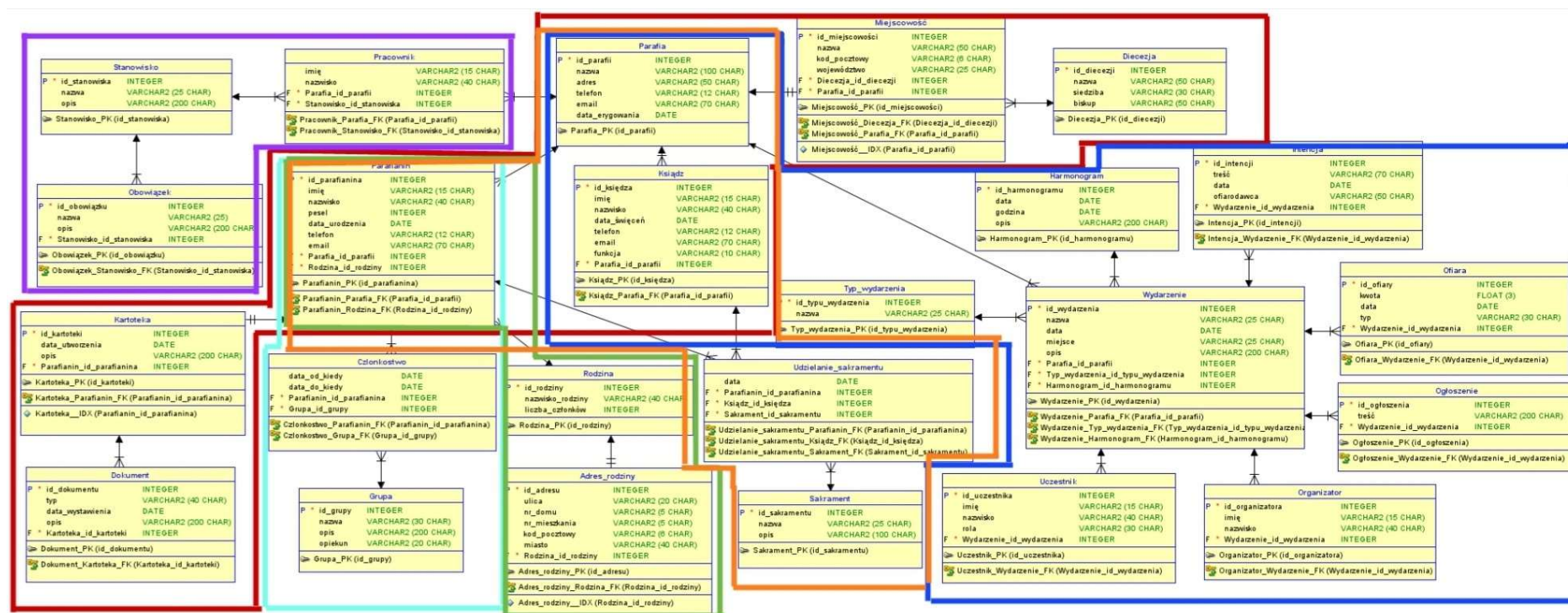


Rysunek 1. Diagram ERD



Rysunek 2. Diagram relacyjny

Podział na konteksty:

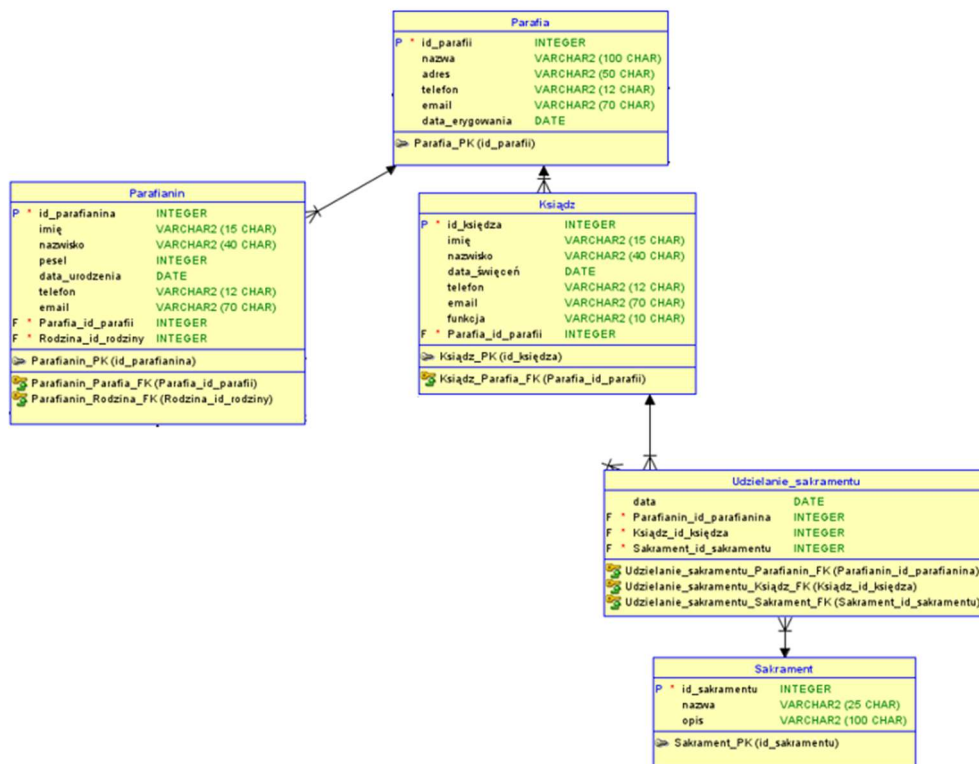


Rysunek 3. Diagram relacyjny po podziale na konteksty

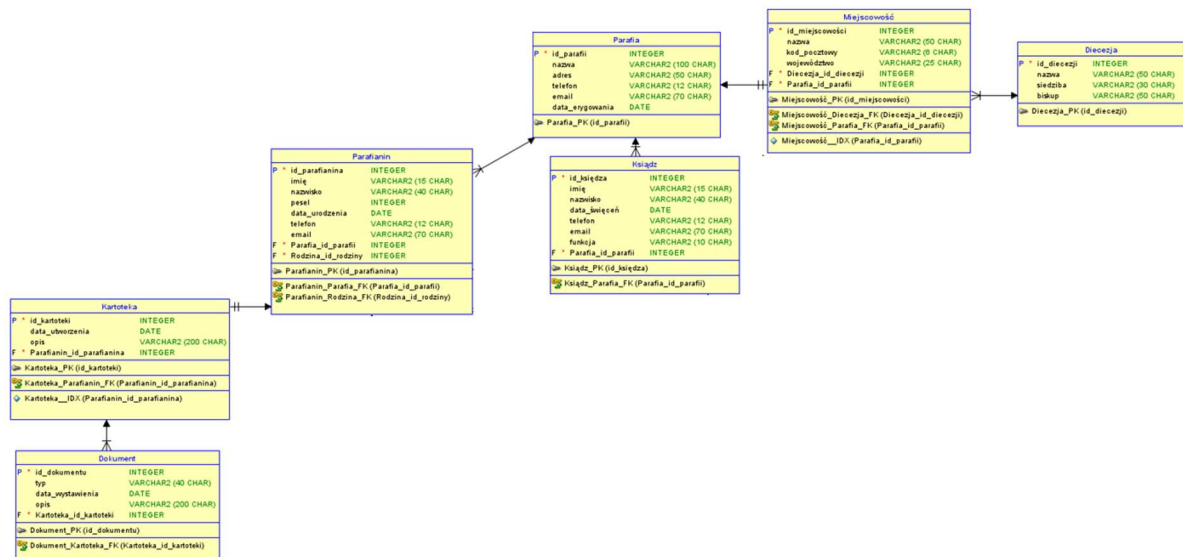
Gdzie:

1. **pomarańczowy** to dziedzina główna (posługa sakramentalna) – encje: Parafia, Parafianin, Ksiądz, Udzielanie sakramentu, Sakrament;
2. **czerwony** to informacje o parafii – encje: Parafia, Miejscowość, Diecezja, Ksiądz, Parafianin, Kartoteka, Dokument;
3. **fioletowy** to organizacja ról i zadań – encje: Pracownik, Stanowisko, Obowiązek;
4. **niebieski** to koordynacja wydarzeń – encje: Parafia, Ksiądz, Wydarzenie, Typ wydarzenia, Harmonogram, Intencja, Ofiara, Ogłoszenie, Uczestnik, Organizator;
5. **zielony** to duszpasterstwo wiernych – Parafianin, Rodzina, Adres rodziny;
6. **błękitny** to grupy parafialne – Parafianin, Członkostwo, Grupa.

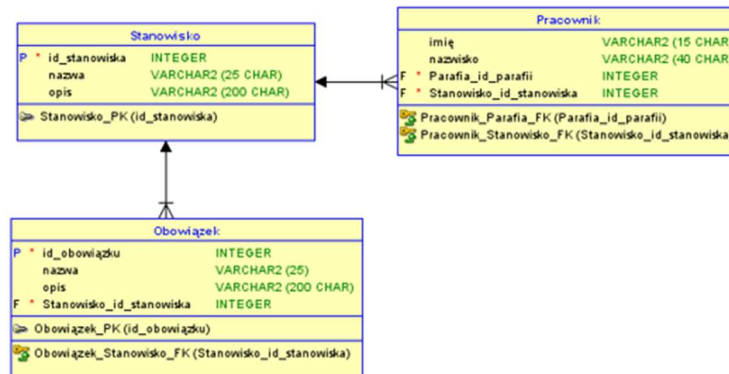
Dziedzina główna (posługa sakramentalna)



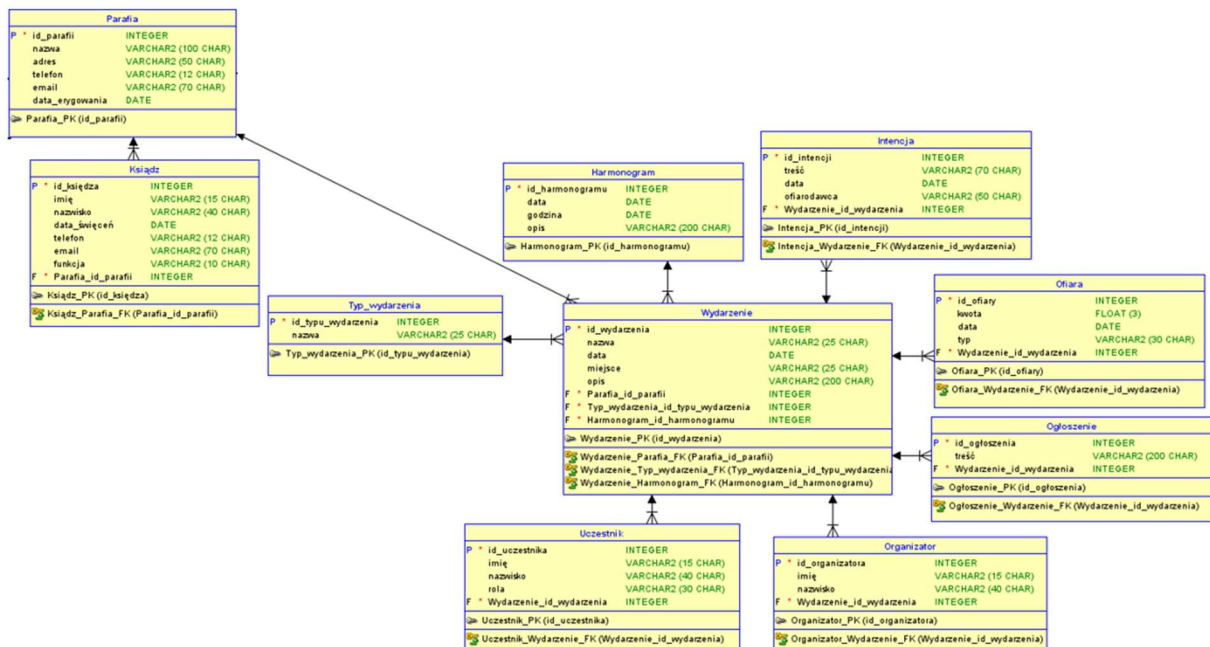
Dziedzina informacji o parafii



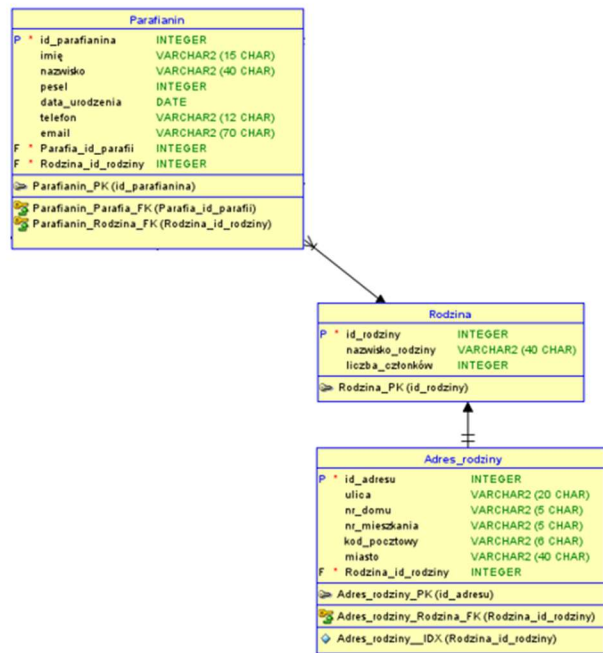
Dziedzina organizacja ról i zadań



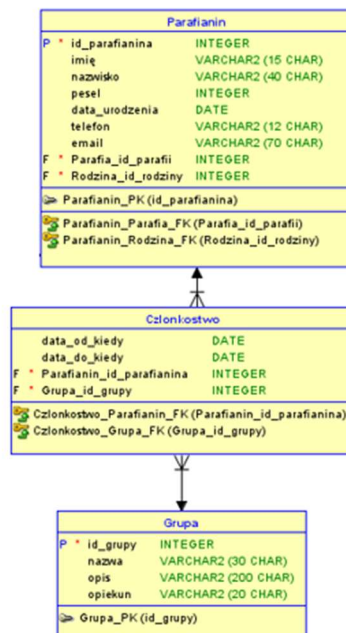
Dziedzina koordynacja wydarzeń



Dziedzina duszpasterstwo wiernych



Dziedzina grupy parafialne



2.2.3 Wyjaśnienie modelu

Pomiędzy poszczególnymi encjami występują następujące relacje:

Relacja	Opis
Parafia – Pracownik (1:N)	Parafia ma jednego lub wielu pracowników, a konkretny pracownik jest przypisany do konkretnej parafii.
Pracownik – Stanowisko (1:N)	Konkretnie stanowisko może mieć jednego lub więcej pracowników, a jeden pracownik jest przypisany do konkretnego stanowiska.
Stanowisko – Obowiązek (1:N)	Konkretnie stanowisko może mieć wiele obowiązków do wykonania, a konkretny obowiązek jest przypisany do jednego stanowiska.
Parafia – Parafianin (1:N)	Parafia ma wielu parafian, a konkretny parafian należy do konkretnej parafii.
Parafianin – Członkostwo (1:N)	Parafianin może należeć do zero, jednej lub wielu grup parafialnych, a konkretne członkostwo ma jednego parafianina.
Grupa – Członkostwo (1:N)	Grupa może mieć wiele członkostw, a konkretne członkostwo należy do jednej grupy.
Rodzina – Parafianin (1:N)	Konkretna rodzina może mieć wielu parafian, a konkretny parafianin należy do jednej rodziny.
Adres_rodziny – Rodzina (1:1)	Konkretna rodzina ma jeden konkretny adres, a konkretny adres jest przypisany do jednej konkretnej rodziny.
Kartoteka – Parafianin (1:1)	Konkretna kartoteka należy do jednego parafianina, a konkretny parafianin ma jedną konkretną kartotekę.
Kartoteka – Dokument (1:N)	Konkretna kartoteka może mieć wiele dokumentów, a konkretny dokument należy do jednej konkretnej kartoteki.
Parafia – Miejscowość (1:1)	Konkretna parafia znajduje się w konkretnej miejscowości, a konkretna miejscowość ma konkretną parafię.
Diecezja – Miejscowość (1:N)	Konkretna diecezja może zawierać wiele miejscowości, a konkretna miejscowość należy do jednej konkretnej diecezji.
Parafia – Ksiądz (1:N)	Konkretna parafia może mieć wielu księży, a konkretny ksiądz należy do jednej, konkretnej parafii.
Ksiądz – Udzielanie sakramentu (1:N)	Konkretny ksiądz może udzielać wielu sakramentów, a konkretny sakrament jest udzielany przez jednego konkretnego księdza.
Sakrament – Udzielanie sakramentu (1:N)	Konkretny sakrament może być udzielany wiele razy, a konkretne udzielenie sakramentu dotyczy jednego konkretnego sakramentu.
Parafia – Wydarzenie (1:N)	Konkretna parafia może mieć wiele wydarzeń, a konkretne wydarzenie dotyczy konkretnej parafii.
Typ_wydarzenia – Wydarzenie (1:N)	Konkretny typ wydarzenia może być przypisany do wielu wydarzeń, a konkretne wydarzenie to jeden konkretny typ wydarzenia (np. msza, rekolekcje).

Harmonogram – Wydarzenie (1:N)	Konkretny harmonogram może być w ramach wielu wydarzeń, a konkretne wydarzenie ma jeden konkretny harmonogram.
Wydarzenie – Intencja (1:N)	Konkretno wydarzenie może być odprowadzane w ramach wielu intencji, a konkretna intencja dotyczy jednego konkretnego wydarzenia.
Wydarzenie – Ofiara (1:N)	Konkretno wydarzenie może mieć zbieranych wiele ofiar, a konkretna ofiara dotyczy konkretnego wydarzenia.
Wydarzenie – Ogłoszenie (1:N)	Konkretno wydarzenie może mieć wiele ogłoszeń, a konkretno ogłoszenie dotyczy konkretnego wydarzenia.
Wydarzenie – Organizator (1:N)	Konkretno wydarzenie może mieć wielu organizatorów, a konkretny organizator jest przypisany do jednego konkretnego wydarzenia.
Wydarzenie – Uczestnik (1:N)	Konkretno wydarzenie może mieć wielu uczestników, a konkretny uczestnik jest przypisany do jednego konkretnego wydarzenia.

Tabela 2. Opis relacji

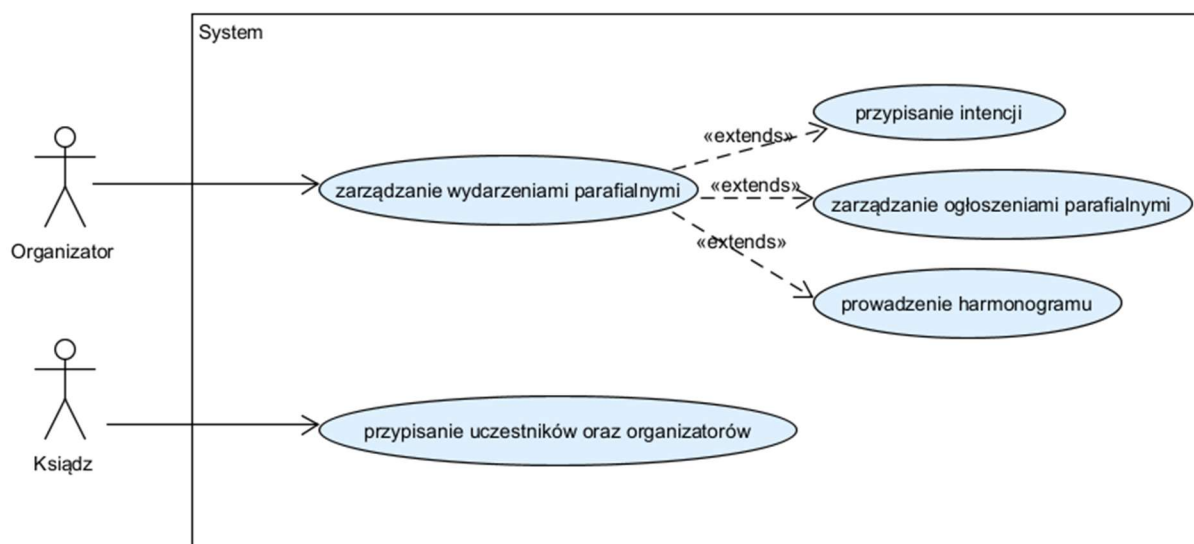
2.3 Model przypadków użycia

2.3.1 Odkrywanie przypadków użycia i aktorów

Dla systemu wspomagającego pracę hodowli królików zostało zaprojektowanych i stworzonych sześć diagramów przypadków użycia.

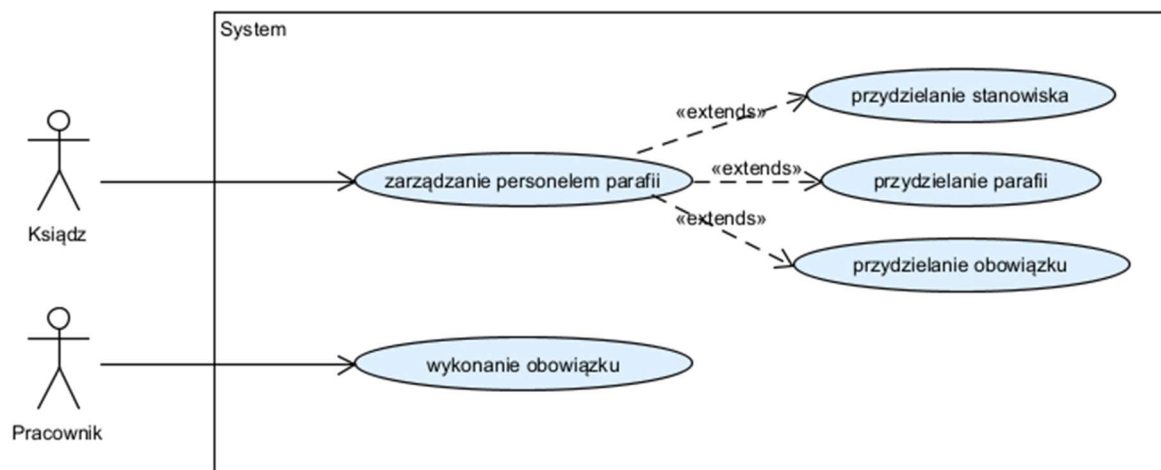
2.3.2 Diagram przypadków użycia

Diagram przypadków użycia – organizowanie wydarzeń parafialnych



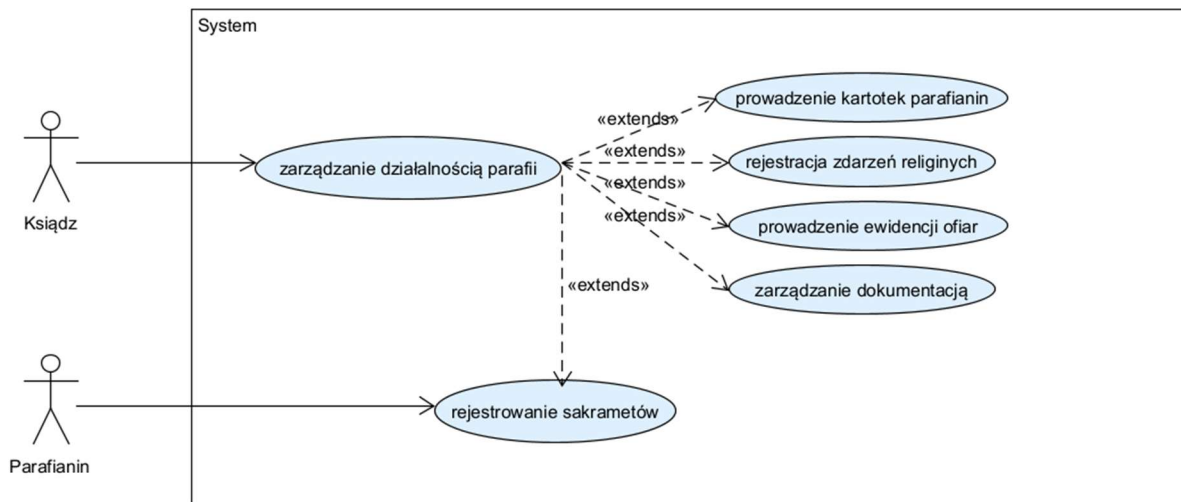
Rysunek 4. Diagram przypadków użycia - organizowanie wydarzeń parafialnych

Diagram przypadków użycia – zarządzanie obowiązkami parafialnymi



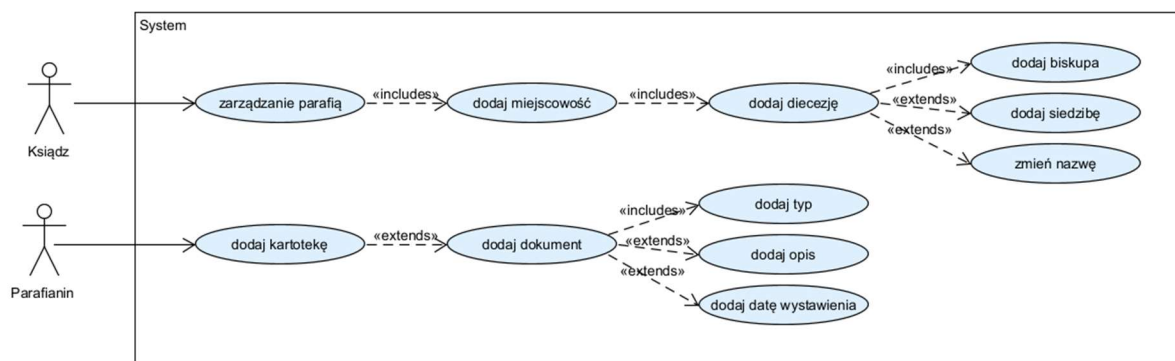
Rysunek 5. Diagram przypadków użycia – zarządzanie obowiązkami parafialnymi

Diagram przypadków użycia – zarządzanie parafianami:



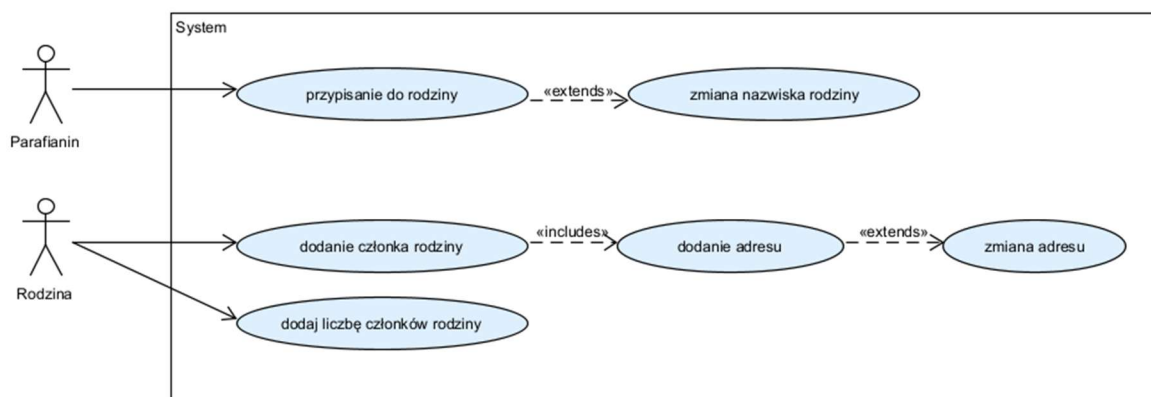
Rysunek 6. Diagram przypadków użycia – zarządzanie parafianami

Diagram przypadków użycia – zarządzanie parafią



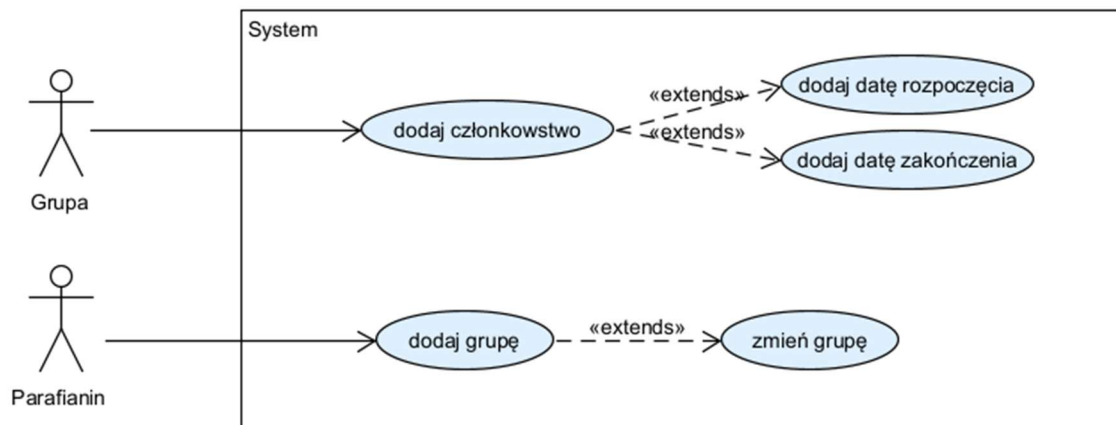
Rysunek 7 Diagram przypadków użycia – zarządzanie parafią

Diagram przypadków użycia – zarządzanie wspólnotą parafialną



Rysunek 8 Diagram przypadków użycia - zarządzanie wspólnotą parafialną

Diagram przypadków użycia – zarządzanie grupami parafialnymi



Rysunek 9 Diagram przypadków użycia - zarządzanie grupami parafialnymi

2.3.3 Opis przypadków użycia

Diagram przypadków użycia – organizowanie wydarzeń parafialnych

Diagram przedstawia przypadki użycia systemu wspierającego zarządzanie wydarzeniami parafialnymi. W systemie wyróżniono dwóch aktorów: **Organizator** oraz **Ksiądz**, którzy korzystają z różnych funkcjonalności.

Głównym przypadkiem użycia jest **zarządzanie wydarzeniami parafialnymi**, z którego korzysta Organizator. Funkcja ta obejmuje kompleksową obsługę wydarzeń odbywających się w parafii. W jej ramach wyróżniono trzy rozszerzenia (relacje «extends»), które reprezentują dodatkowe, opcjonalne funkcjonalności:

- przypisanie intencji,
- zarządzanie ogłoszeniami parafialnymi,
- prowadzenie harmonogramu.

Oznacza to, że podczas zarządzania wydarzeniami Organizator może skorzystać z tych funkcji w zależności od potrzeb.

Drugim przypadkiem użycia jest **przypisanie uczestników oraz organizatorów**, z którego korzysta aktor Ksiądz. Funkcjonalność ta umożliwia zarządzanie osobami zaangażowanymi w wydarzenia parafialne, w tym przypisywanie odpowiednich ról.

System został zaprojektowany w sposób modularny, co pozwala na elastyczne rozszerzanie jego funkcjonalności oraz dostosowanie do różnych potrzeb organizacyjnych parafii.

Diagram przypadków użycia – zarządzanie obowiązkami parafialnymi

Diagram przedstawia przypadki użycia systemu odpowiedzialnego za zarządzanie personelem parafii. Wyróżniono dwóch aktorów: **Ksiądz** oraz **Pracownik**, którzy korzystają z różnych funkcji systemu.

Głównym przypadkiem użycia jest **zarządzanie personelem parafii**, realizowane przez aktora Księdza. Funkcjonalność ta umożliwia kompleksowe administrowanie personelem, w tym przypisywanie ról i obowiązków. W ramach tego przypadku użycia wyróżniono trzy relacje rozszerzenia («extends»), które reprezentują dodatkowe operacje:

- przydzielanie stanowiska,
- przydzielanie parafii,
- przydzielanie obowiązku.

Relacje te wskazują, że poszczególne działania mogą być wykonywane opcjonalnie w zależności od aktualnych potrzeb zarządzania personelem.

Drugim przypadkiem użycia jest **wykonanie obowiązku**, z którego korzysta aktor Pracownik. Funkcja ta pozwala na realizację przypisanych zadań przez członków personelu, co stanowi końcowy etap procesu zarządzania obowiązkami.

Diagram odzwierciedla hierarchiczną strukturę zarządzania, gdzie Ksiądz pełni rolę administratora, a Pracownik realizuje powierzone zadania. Takie podejście umożliwia przejrzyste rozdzielenie odpowiedzialności w systemie.

Diagram przypadków użycia – zarządzanie parafianami:

Diagram przedstawia funkcjonalności systemu związane z zarządzaniem działalnością parafii. Wyróżniono dwóch aktorów: **Ksiądz** oraz **Parafianin**, którzy korzystają z systemu w różnym zakresie.

Głównym przypadkiem użycia jest **zarządzanie działalnością parafii**, realizowane przez aktora Księdza. Funkcjonalność ta obejmuje szeroki zakres działań administracyjnych i organizacyjnych związanych z funkcjonowaniem parafii. W jej ramach wyróżniono kilka relacji rozszerzenia («extends»), które reprezentują szczegółowe operacje:

- prowadzenie kartotek parafian,
- rejestracja zdarzeń religijnych,
- prowadzenie ewidencji ofiar,
- zarządzanie dokumentacją.

Każda z tych funkcji może być wykonywana w zależności od aktualnych potrzeb, co wskazuje na elastyczność systemu i jego modułową budowę.

Dodatkowo w diagramie uwzględniono przypadek użycia **rejestrowanie sakramentów**, który jest powiązany z główną funkcjonalnością poprzez relację «extends». Oznacza to, że proces ten stanowi rozszerzenie zarządzania działalnością parafii i może być realizowany w określonych sytuacjach.

Aktor Parafianin korzysta z funkcji rejestrowania sakramentów, co wskazuje na jego udział w procesach religijnych obsługiwanych przez system. Ksiądz natomiast pełni rolę administratora i nadzoruje całość działalności parafii.

Diagram obrazuje kompleksowe podejście do zarządzania parafią, integrując zarówno aspekty administracyjne, jak i religijne w jednym systemie.

Diagram przypadków użycia – zarządzanie parafią

Diagram przedstawia przypadki użycia systemu wspierającego zarządzanie parafią. W systemie wyróżniono dwóch aktorów: Ksiądz oraz Parafianin, którzy korzystają z różnych funkcjonalności systemu.

Głównym przypadkiem użycia jest zarządzanie parafią, realizowane przez aktora Księdza. Funkcjonalność ta umożliwia administrowanie podstawowymi danymi parafii oraz jej strukturą administracyjną. W ramach tego przypadku użycia występuje relacja włączenia («includes») do:

- dodaj miejscowość,

która z kolei zawiera relację «includes» do:

- dodaj diecezję.

Oznacza to, że w procesie zarządzania parafią wymagane jest określenie miejscowości oraz powiązanej z nią diecezji.

Przypadek użycia dodaj diecezję posiada dodatkowe relacje rozszerzenia («extends»), które reprezentują opcjonalne funkcjonalności:

- dodaj biskupa,
- dodaj siedzibę,
- zmień nazwę.

Relacje te wskazują, że podczas dodawania diecezji możliwe jest uzupełnienie jej danych o dodatkowe informacje w zależności od potrzeb.

Drugim obszarem funkcjonalnym systemu jest zarządzanie kartoteką parafianina, z którego korzysta aktor Parafianin. Głównym przypadkiem użycia jest:

- dodaj kartotekę.

Przypadek ten posiada relację rozszerzenia («extends») do:

- dodaj dokument,

co oznacza, że dodanie dokumentu stanowi opcjonalne rozszerzenie procesu tworzenia kartoteki.

Przypadek użycia dodaj dokument zawiera relację włączenia («includes») do:

- dodaj typ,

oraz relację rozszerzenia («extends») do dodatkowych operacji:

- dodaj opis,
- dodaj datę wystawienia.

Oznacza to, że określenie typu dokumentu jest wymagane, natomiast pozostałe informacje mogą być uzupełniane opcjonalnie.

Diagram przedstawia modularną strukturę systemu, w której główne funkcjonalności mogą być rozwijane o dodatkowe operacje. Takie podejście zapewnia elastyczność w zarządzaniu danymi parafii oraz kartotekami parafian, a także umożliwia łatwe dostosowanie systemu do zmieniających się potrzeb.

Diagram przypadków użycia – zarządzanie wspólnotą parafialną

Diagram przedstawia przypadki użycia systemu wspierającego **zarządzanie wspólnotą parafialną**. W systemie wyróżniono dwóch aktorów: **Parafianin** oraz **Rodzina**, którzy korzystają z dostępnych funkcjonalności systemu.

Głównym przypadkiem użycia realizowanym przez aktora Parafianina jest:

- przypisanie do rodziny.

Funkcjonalność ta umożliwia powiązanie parafianina z wybraną rodziną w systemie.

Przypadek ten posiada relację rozszerzenia («extends») do:

- zmiana nazwiska rodziny.

Oznacza to, że w trakcie przypisywania parafianina do rodziny istnieje możliwość opcjonalnej zmiany nazwiska rodziny, jeśli zaistnieje taka potrzeba.

Drugim obszarem funkcjonalnym systemu jest zarządzanie rodziną, realizowane przez aktora Rodzina. Głównym przypadkiem użycia jest:

- dodanie członka rodziny.

Funkcjonalność ta umożliwia rozszerzenie składu rodziny o nowych członków. Przypadek ten zawiera relację włączenia («includes») do:

- dodanie adresu,

co oznacza, że podczas dodawania członka rodziny wymagane jest określenie adresu.

Przypadek użycia **dodanie adresu** posiada relację rozszerzenia («extends») do:

- zmiana adresu.

Relacja ta wskazuje, że po dodaniu adresu istnieje możliwość jego późniejszej modyfikacji.

Dodatkowym przypadkiem użycia realizowanym przez aktora Rodzina jest:

- dodaj liczbę członków rodziny.

Funkcjonalność ta pozwala na określenie liczby członków należących do danej rodziny.

Diagram przedstawia podstawowe operacje związane z zarządzaniem wspólnotą parafialną, obejmujące zarówno przypisywanie parafian do rodzin, jak i zarządzanie strukturą oraz danymi rodzin. Modułarna budowa systemu umożliwia jego dalszą rozbudowę oraz dostosowanie do zmieniających się potrzeb.

Diagram przypadków użycia – zarządzanie grupami parafialnymi

Diagram przedstawia przypadki użycia systemu wspierającego **zarządzanie grupami parafialnymi**. W systemie wyróżniono dwóch aktorów: **Grupa** oraz **Parafianin**, którzy korzystają z dostępnych funkcjonalności.

Głównym przypadkiem użycia realizowanym przez aktora Grupa jest:

- dodaj członkostwo.

Funkcjonalność ta umożliwia przypisanie parafianina do wybranej grupy parafialnej. Przypadek ten posiada dwie relacje rozszerzenia («extends»), które reprezentują opcjonalne operacje:

- | | | |
|---------------------------|------|--------------|
| dodaj | datę | rozpoczęcia, |
| • dodaj datę zakończenia. | | |

Oznacza to, że podczas dodawania członkostwa można dodatkowo określić okres przynależności parafianina do grupy, jednak nie jest to wymagane.

Drugim obszarem funkcjonalnym systemu jest zarządzanie samymi grupami, realizowane przez aktora Parafianin. Głównym przypadkiem użycia jest:

- dodaj grupę.

Funkcjonalność ta umożliwia utworzenie nowej grupy parafialnej w systemie. Przypadek ten posiada relację rozszerzenia («extends») do:

- zmień grupę.

Relacja ta wskazuje, że po utworzeniu grupy istnieje możliwość jej późniejszej modyfikacji.

Diagram przedstawia podstawowe operacje związane z zarządzaniem grupami parafialnymi, obejmujące zarówno przypisywanie członków do grup, jak i tworzenie oraz edycję grup. Modułarna struktura systemu pozwala na jego dalszą rozbudowę oraz elastyczne dostosowanie do potrzeb organizacyjnych parafii.

3 Projekt

4 Implementacja

System eParafia został zrealizowany jako aplikacja backendowa w języku Java 21 z wykorzystaniem frameworka Spring Boot 4.0.6. Persystencja danych oparta jest na Spring Data JPA z implementacją Hibernate 7 oraz bazą danych H2 w trybie plikowym (plik `./data/eparish`). Do generowania kodu szablonowego encji i serwisów użyto biblioteki Lombok. Projekt jest budowany narzędziem Gradle (wrapper 9.4). Dokumentacja interaktywna API udostępniana jest przez bibliotekę springdoc-openapi 2.8.9 pod adresem `/swagger-ui/index.html`. Serializacja JSON realizowana jest przez Jackson 3 (pakiet `tools.jackson`).

4.1 Architektura aplikacji

Projekt stosuje podejście Domain-Driven Design (DDD) z czystym podziałem na cztery warstwy. Kontrolery (pakiet `api`) pełnią wyłącznie rolę mapowania żądań HTTP na obiekty poleceń (Command Records) i odpowiedzi JSON – nie zawierają żadnej logiki biznesowej. Serwisy (pakiet `application`) realizują przypadki użycia, walidują dane wejściowe, obsługują wyjątki i orkiestrują operacje na fabrykach i repozytoriach. Fabryki (pakiet `application`) odpowiadają za tworzenie nowych obiektów domenowych, oddzielając logikę konstrukcji od logiki serwisowej. Repozytoria (pakiet `domain`) są interfejsami Spring Data JPA zapewniającymi persystencję encji poprzez Hibernate.

4.2 Konteksty domenowe

Aplikacja podzielona jest na sześć oddzielnych kontekstów (bounded contexts), z których każdy stanowi osobny pakiet Java zawierający własne encje JPA, repozytoria, fabrykę i serwis aplikacyjny. Kontekst `eventcoordination` (`EventCoordinationService`, `EventFactory`) obsługuje zarządzanie wydarzeniami parafialnymi, harmonogramami, intencjami, ogłoszeniami, ofiarami, uczestnikami i organizatorami. Kontekst `parishinformation` (`ParishInformationService`, `ParishInfoFactory`) zarządza strukturą parafii, diecezjami, miejscowościami, kartotekami parafian i dokumentami. Kontekst `pastoralcare` (`PastoralCareService`, `PastoralCareFactory`) realizuje duszpasterstwo wiernych: zarządzanie parafianami, rodzinami i adresami rodzin. Kontekst `parishgroups` (`ParishGroupService`, `ParishGroupFactory`) obsługuje grupy parafialne i członkostwa. Kontekst `staffmanagement` (`ParishOperationsService`, `StaffFactory`) zarządza personelem parafii, stanowiskami i obowiązkami. Kontekst `sacramentalministry` (`SacramentalMinistryService`, `SacramentalMinistryFactory`) rejestruje sakramenty i ich udzielanie parafianom przez kapłanów.

4.3 Encje JPA i persystencja danych

Model danych obejmuje 24 encje JPA, spełniając wymaganie projektowe 20 encji z nadwyżką. Encje zmapowane są na tabele relacyjnej bazy H2 poprzez adnotacje JPA: `@Entity`, `@Table`, `@Column`, `@ManyToOne`, `@OneToOne` oraz `@OneToMany`. Schemat bazy jest tworzony i aktualizowany automatycznie przez Hibernate (`ddl-auto=update`). Kod szablonowy encji (getterzy, setterzy, konstruktory) generowany jest przez bibliotekę Lombok z adnotacją `@Data`. Baza H2 pracuje w trybie plikowym (`jdbc:h2:file:./data/eparish`), co zapewnia trwałość danych

między uruchomieniami aplikacji. Konsola zarządzania H2 dostępna jest pod adresem `http://localhost:8080/h2-console`.

Przy każdym starcie aplikacji plik `data.sql` (`spring.sql.init.mode=always`) ładuje dane startowe: diecezję, miejscowość, parafię, personel (2 księży, pracownika), dwa wydarzenia z harmonogramem, intencje, ogłoszenia, ofiary, uczestników, organizatorów, sześć grup parafialnych, siedem sakramentów (Chrzest, Bierzmowanie, Eucharystia, Pokuta, Namaszczenie, Kapłaństwo, Małżeństwo), trzy rodziny z adresami, trzech parafian z kartotekami i dokumentami oraz przykłady udzielania sakramentów. Flaga `spring.sql.init.continue-on-error=true` umożliwia ponowny start aplikacji bez błędów, gdy dane startowe już istnieją w bazie.

4.4 Agregaty domenowe

W architekturze DDD zaimplementowano trzy agregaty domenowe, które grupują powiązane encje i udostępniają metody logiki domeny. Agregat `EventAggregate` (złożony) ma jako korzeń encję `ParishEvent` i łączy z nią encje: `Intention`, `Announcement`, `Offering`, `Participant` i `Organizer`. Udostępnia metody domeny: `totalOfferings()` obliczającą sumę zebranych ofiar, `realizedIntentionCount()` i `plannedIntentionCount()` zwracające statystyki intencji, `totalEngaged()` sumującą uczestników i organizatorów, `announcementCount()` zwracającą liczbę ogłoszeń oraz `isIntentionAlreadyAssigned()` sprawdzającą duplikaty intencji. Agregat dostępny jest przez endpoint `GET /api/events/{id}/aggregate`.

Agregat `ParishionerAggregate` grupuje parafianina (`Parishioner`) z jego kartoteką (`ParishRecord`) i dokumentami (`Document`), zwracając pełny profil wiernego. Dostępny przez `GET /api/parishioners/{id}/aggregate`. Agregat `ParishGroupAggregate` łączy grupę parafialną (`ParishGroup`) z listą jej członkostw (`Membership`). Dostępny przez `GET /api/groups/{id}/aggregate`. Każdy agregat jest niemutowalnym obiektem Java (final class) z kopiami obronnymi list (`List.copyOf`), co chroni spójność stanu agregatu.

4.5 Interfejs REST API

Aplikacja udostępnia interfejs REST z prefiksem `/api` obsługiwany przez pięć kontrolerów tematycznych: `EventCoordinationController`, `ParishInformationController`, `PastoralCareController`, `ParishOperationsController` oraz `SacramentalMinistryController`. Każdy zasób listowy obsługuje metody HTTP: GET (z opcjonalnym filtrowaniem po parametrach zapytania), POST (tworzenie zasobu, odpowiedź 201 Created), PATCH (częściowa aktualizacja), PUT (pełna aktualizacja) oraz DELETE (usuwanie, odpowiedź 204 No Content). Łącznie zaimplementowano ponad 20 przypadków użycia realizowanych przez dedykowane endpointy.

Filtrowanie list zaimplementowano w klasie pomocniczej `ListFilterSupport`. Metoda `filter()` przyjmuje listę predykatów zbudowanych z parametrów zapytania (query params). Predykat tworzony jest tylko dla tych parametrów, które faktycznie zostały przekazane w żądaniu; pominięcie parametru oznacza brak filtrowania po tym kryterium. Wszystkie parametry filtrowania są opcjonalne i można je łączyć w logikę AND. Przykład: `GET /api/announcements?eventId=1` zwraca ogłoszenia wyłącznie dla wydarzenia o `id=1`, natomiast `GET /api/announcements` bez parametrów zwraca pełną listę.

Częściowe aktualizacje (PATCH) realizowane są przez klasę `PatchBodySupport`. Ciało żądania deserializowane jest jako `Map<String, Object>`, co pozwala odróżnić pole pominięte w żądaniu (klucz nieobecny w mapie – nie zmieniaj) od jawnie przekazanego null (klucz obecny, wartość

null – odepnij relację). To rozróżnienie jest istotne np. przy odpinaniu parafianina od rodziny: PATCH /api/parishioners/{id} z {"familyId": null} odpina parafianina, natomiast brak klucza familyId pozostawia przypisanie bez zmian. PatchBodySupport korzysta z Jackson 3 (pakiet tools.jackson), będącego domyślnym serializatorem w Spring Boot 4.

4.6 Obsługa błędów

Obsługa błędów scentralizowana jest w klasie ApiExceptionHandler oznaczonej adnotacją @ControllerAdvice. Serwisy rzucają wyjątek ResponseStatusException z odpowiednim kodem HTTP: HttpStatus.NOT_FOUND (404) gdy zasób nie istnieje w bazie, HttpStatus.BAD_REQUEST (400) przy nieprawidłowych danych wejściowych (np. brakujące wymagane pole familyId przy adresie rodziny) oraz HttpStatus.CONFLICT (409) przy próbie naruszenia unikalności (np. zduplikowany adres rodziny). Handler mapuje te wyjątki na ustrukturyzowane odpowiedzi JSON, ułatwiając diagnostykę po stronie klienta API.

4.7 Inicjalizacja i konfiguracja

Konfiguracja aplikacji przechowywana jest w pliku src/main/resources/application.properties. Baza H2 skonfigurowana jest w trybie plikowym (spring.datasource.url=jdbc:h2:file:./data/eparish), dzięki czemu dane zachowują się między restartami serwera. Opcja spring.jpa.defer-datasource-initialization=true zapewnia, że schemat jest tworzony przez Hibernate zanim zostanie wykonany skrypt data.sql. Katalog data/ jest wykluczony z repozytorium Git wpisem w .gitignore, co zapobiega przypadkowemu commitowaniu danych testowych. Aby zresetować bazę do stanu startowego wystarczy usunąć katalog data/ i ponownie uruchomić aplikację.

4.8 Dokumentacja interaktywna API

Interaktywna dokumentacja wszystkich endpointów dostępna jest przez Swagger UI pod adresem <http://localhost:8080/swagger-ui/index.html>. Dokumentacja generowana jest automatycznie przez bibliotekę springdoc-openapi 2.8.9 na podstawie adnotacji @Operation (opis endpointu) i @Tag (grupowanie w sekcje tematyczne) umieszczonych w kontrolerach REST. Swagger UI umożliwia bezpośrednie testowanie każdego endpointu z poziomu przeglądarki bez potrzeby używania zewnętrznych narzędzi. Strona startowa aplikacji (GET /) zawiera listę wszystkich zaimplementowanych przypadków użycia wraz z metodami HTTP i ścieżkami.

4.9 Ciągła integracja (GitHub Actions)

Repozytorium projektu wyposażone jest w potok ciągłej integracji (CI) zdefiniowany w pliku .github/workflows/ci.yml przy użyciu platformy GitHub Actions. Workflow nosi nazwę CI i uruchamiany jest automatycznie przy każdym wypchnięciu kodu (push) na gałąź main oraz przy każdym Pull Requesście skierowanym do tej gałęzi. Dzięki temu każda zmiana w kodzie jest natychmiast weryfikowana, co zapobiega włączaniu regresji do głównej linii projektu.

Workflow składa się z jednego zadania (job) o nazwie test, wykonywanego na wirtualnej maszynie ubuntu-latest. Przebieg zadania obejmuje cztery kroki. Pierwszy krok pobiera kod źródłowy z repozytorium (actions/checkout@v4). Drugi krok konfiguruje środowisko Java 21 w dystrybucji Eclipse Temurin (actions/setup-java@v4). Trzeci krok cachuje pobrane zależności Gradle (katalogi ~/.gradle/caches i ~/.gradle/wrapper) przy użyciu

actions/cache@v4, z kluczem opartym na skrócie plików build.gradle i gradle-wrapper.properties, co znacznie skraca czas kolejnych uruchomień. Czwarty krok uruchamia testy poleceniem ./gradlew test --no-daemon. Po zakończeniu testów – niezależnie od ich wyniku (if: always()) – raport HTML generowany przez JUnit (katalog build/reports/tests/test/) jest archiwizowany jako artefakt (actions/upload-artifact@v4) pod nazwą test-report, skąd można go pobrać bezpośrednio z poziomu GitHub.

4.10 Testy automatyczne (JUnit + MockMvc)

Testy integracyjne zaimplementowane są w klasie ApiIntegrationTest (src/test/java/edu/prz/eparish/api/). Klasa oznaczona jest adnotacjami @SpringBootTest(webEnvironment = RANDOM_PORT) oraz @AutoConfigureMockMvc, co uruchamia pełny kontekst aplikacji Spring na losowym porcie i wstrzykuje instancję MockMvc do wykonywania żądań HTTP w testach. Profil @ActiveProfiles("test") pozwala na oddzielną konfigurację środowiska testowego.

Zestaw zawiera 9 scenariuszy testowych pokrywających główne przepływy systemu: homeReturnsApiInfo weryfikuje, że strona startowa GET / zwraca obiekt JSON z polem apiBase równym "/api"; listParishesReturnsSeedData sprawdza, że GET /api/parishes zwraca dane startowe (parafia o id=1); seedEventExistsForIntentionAndAnnouncementTests potwierdza obecność zdarzenia startowego wymaganego przez kolejne testy; addFamilyMatchesBrunoTest tworzy rodzinę Kowalscy przez POST /api/families i weryfikuje odpowiedź 201 z poprawnymi polami; createGroupMatchesBrunoTest tworzy grupę Oaza Młodzieżowa przez POST /api/groups; addEmployeeMatchesBrunoTest dodaje pracownika przez POST /api/employees i sprawdza powiązanie z parafią; addIntentionMatchesBrunoTest dodaje intencję przez POST /api/intentions i weryfikuje obecność wygenerowanego id oraz poprawne eventId; addAnnouncementMatchesBrunoTest dodaje ogłoszenie przez POST /api/announcements; registerSacramentMatchesBrunoTest rejestruje udzielenie sakramentu przez POST /api/sacrament-administrations i weryfikuje powiązanie z parafianinem.

Testy uruchamiane są poleceniem .\gradlew.bat test. Każdy scenariusz wykonuje realne żądanie HTTP przez MockMvc do działającego kontekstu aplikacji i sprawdza zarówno kod statusu HTTP (201 Created lub 200 OK), jak i strukturę zwróconego obiektu JSON przy użyciu asercji jsonPath. Raporty HTML z wynikami testów generowane są w katalogu build/reports/tests/test/ i archiwizowane automatycznie przez pipeline GitHub Actions.

4.11 Repozytorium i wersjonowanie

Kod źródłowy projektu przechowywany jest w publicznym repozytorium Git na platformie GitHub pod adresem <https://github.com/Slasq/eParafia>. Projekt prowadzony jest na jednej gałęzi głównej (main), do której trafiają wszystkie zmiany. Historia commitów odzwierciedla kolejne etapy prac: anglikację kodu (nazwy pakietów, klas, pól encji i tabel bazy danych), implementację operacji PATCH i DELETE, filtrowanie list GET, naprawę obsługi częściowej aktualizacji pól relacyjnych oraz rozbudowę schematu o nowe encje i agregaty. Katalogi z danymi lokalnej bazy (data/) i plikami tymczasowymi pakietu Word (pliki zaczynające się od ~\$) są wykluczone z wersjonowania wpisami w pliku .gitignore.

5 Testy

5.1 Cel testów

Celem testów było sprawdzenie poprawności działania najważniejszych funkcji systemu eParafia zgodnie z założeniami biznesowymi oraz logiką relacyjną bazy danych opisaną w dokumentacji projektu. Testy obejmowały m.in. obsługę struktury administracyjnej (parafie, diecezje), zarządzanie personelem, ewidencję wspólnoty (parafianie, kartoteki, rodziny), organizację grup parafialnych, a także koordynację wydarzeń i sakramentów.

Testy wykonano za pomocą automatycznych żądań do API REST zdefiniowanych w plikach konfiguracji OpenCollection (.yml) oraz uruchamiano poprzez narzędzie testowe bazujące na wbudowanej bazie danych H2 (.\\gradlew.bat test), wysyłając żądania HTTP do lokalnie uruchomionego backendu aplikacji.

5.2 Organizacja testów

Testy zostały podzielone na foldery odpowiadające głównym obszarom (domenom) działania systemu, zgodnie ze strukturą plików OpenCollection:

- **Parish_Information** (Zarządzanie strukturą parafii)
- **Staff_Management** (Zarządzanie personelem i obowiązkami)
- **Pastoral_Care** (Opieka duszpasterska, rodziny i kartoteki)
- **Parish_Groups** (Zarządzanie grupami parafialnymi)
- **Event_Coordination** (Organizowanie wydarzeń i harmonogramów)
- **Sacramental_Ministry** (Posługa sakramentalna)

✓ Eparafia - testy

✓ Organizowanie wydarzeń parafialnych

- > Zarządzanie wydarzeniami parafialnymi
- > Zarządzanie intencjami
- > Zarządzanie ogłoszeniami parafialnymi
- > Zarządzanie harmonogramem
- > Zarządzanie uczestnikami i organizatorami

✓ Zarządzanie obowiązkami parafialnymi

- > Zarządzanie personelem parafii
- > Zarządzanie stanowiskami
- > Przypisanie pracownika do parafii
- > Zarządzanie obowiązkami

✓ Zarządzanie parafianami

✓ zarządzanie działalnością parafii

- > prowadzenie kartotek parafian
- > rejestracja zdarzeń religijnych
- > prowadzenie ewidencji ofiar
- > zarządzanie dokumentacją
- > rejestrowanie sakramentów

✓ Zarządzanie parafią

- > Zarządzanie parafiami
- > Zarządzanie miejscowościami
- > Zarządzanie diecezjami
- > Zarządzanie kartoteką i dokumentami

✓ Zarządzanie wspólnotą parafialną

- > Przypisanie parafianina do rodziny
- > Zarządzanie rodziną
- > Zarządzanie adresem rodziny

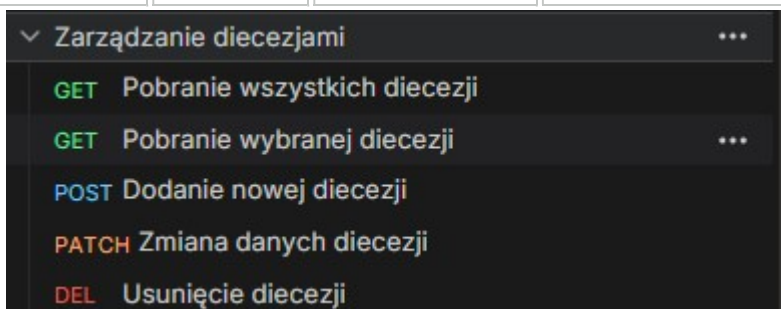
✓ Zarządzanie grupami parafialnymi

- > Zarządzanie grupą parafialną
- > Zarządzanie członkostwem w grupie

5.3 Przygotowanie danych testowych

Przed wykonaniem właściwych scenariuszy, konieczne było zasilenie bazy testowymi danymi początkowymi, wymaganymi do poprawnego mapowania relacji. Dane dodawano przez API.

Lp.	Nazwa testu	Metoda	Endpoint	Cel
1	Dodaj diecezję	POST	/api/dioceses	Utworzenie testowej diecezji
2	Dodaj parafię	POST	/api/parishes	Utworzenie parafii przypisanej do diecezji
3	Dodaj pracownika	POST	/api/staff	Utworzenie profilu księdza/pracownika
4	Dodaj stanowisko	POST	/api/positions	Zdefiniowanie roli (np. Proboszcz)

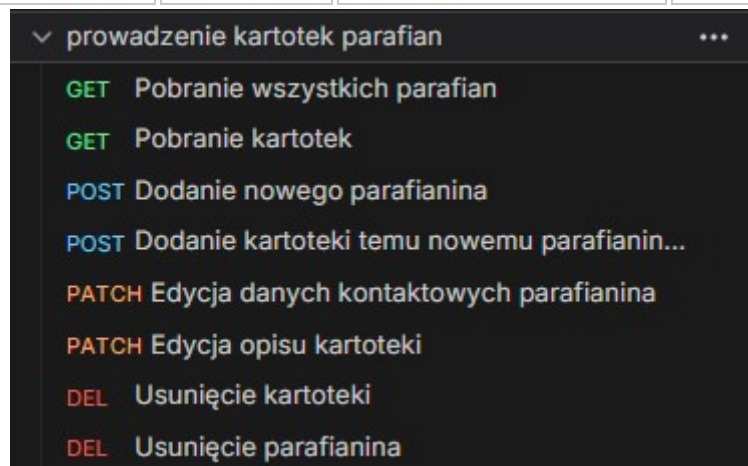


Test polegał na sprawdzeniu działania endpointu odpowiedzialnego za dodawanie nowej struktury terytorialnej do systemu. Wysłano żądanie HTTP metodą POST. Po wykonaniu żądania system poprawnie zapisał dane w bazie danych i zwrócił kod odpowiedzi 200 OK/201 Created. Test potwierdził poprawne działanie endpointu REST API oraz komunikację aplikacji z bazą danych H2.

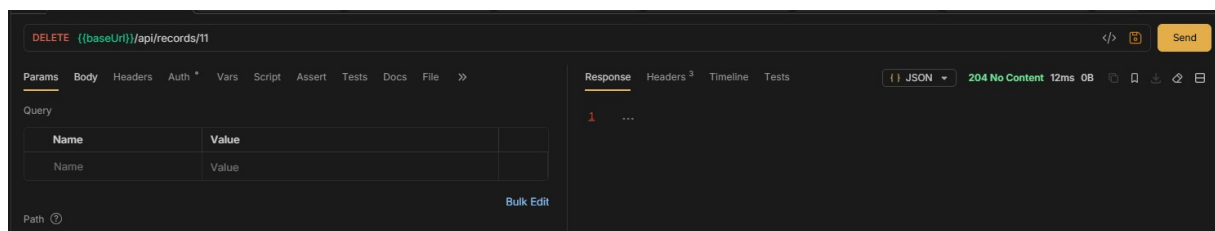
5.4 Testy opieki duszpasterskiej i kartotek (Pastoral Care)

Lp.	Nazwa testu	Metoda	Endpoint	Oczekiwany rezultat
1	Dodaj parafianina	POST	/api/parishioners	System tworzy nowego parafianina
2	Lista parafian	GET	/api/parishioners	System zwraca listę zapisanych parafian

Lp.	Nazwa testu	Metoda	Endpoint	Oczekiwany rezultat
3	Utwórz kartotekę	POST	/api/dossiers	System przypisuje nową kartotekę do parafianina
4	Próba usunięcia parafianina z kartoteką	DELETE	/api/parishioners/{id}	System blokuje operację (błąd 400/409)
5	Usunięcie kartoteki	DELETE	/api/dossiers/{id}	System usuwa kartotekę
6	Usunięcie parafianina (sukces)	DELETE	/api/parishioners/{id}	System poprawnie usuwa parafianina



Test dotyczył sprawdzenia działania endpointu odpowiedzialnego za dodawanie nowego parafianina. Wysłano dane zawierające imię, nazwisko oraz dane kontaktowe. System poprawnie zapisał osobę i zwrócił status 200 OK.



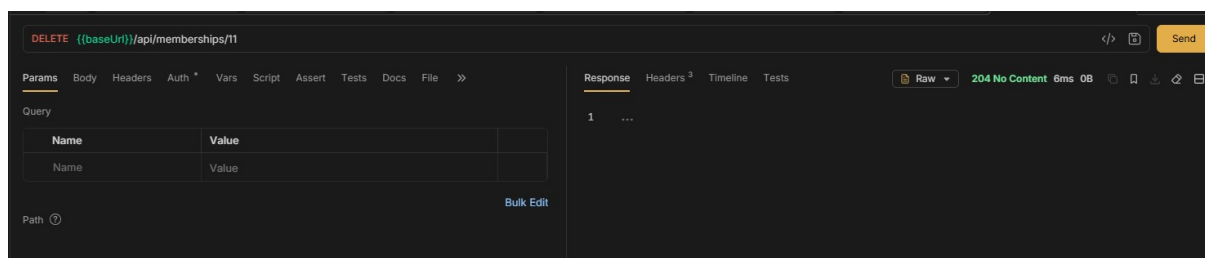
Test integralności usuwania: Endpoint został przetestowany pod kątem rygorystycznej obsługi błędów i relacji referencyjnych. Próba usunięcia parafianina posiadającego aktywną kartotekę została poprawnie zablokowana przez system w celu uniknięcia "osieroconych" rekordów. Właściwe usunięcie parafianina (kod 204 No Content) powiodło się dopiero po wcześniejszym wysłaniu żądania DELETE dla powiązanej z nim kartoteki.

5.5 Testy grup parafialnych (Parish Groups)

Lp.	Nazwa testu	Metoda	Endpoint	Oczekiwany rezultat
1	Utwórz grupę	POST	/api/groups	System tworzy nową grupę parafialną
2	Dodaj członka do grupy	POST	/api/groups/{id}/members	System przypisuje osobę do grupy
3	Próba usunięcia pełnej grupy	DELETE	/api/groups/{id}	System odrzuca żądanie usunięcia grupy z członkami
4	Usunięcie pustej grupy	DELETE	/api/groups/{id}	System poprawnie usuwa grupę



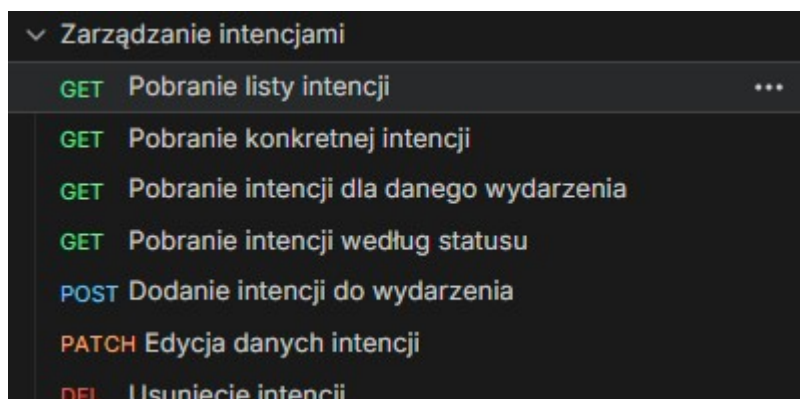
Endpoint służy do utworzenia grupy zrzeszającej wiernych. Test polegał na dodaniu grupy, a następnie przypisaniu do niej członków.



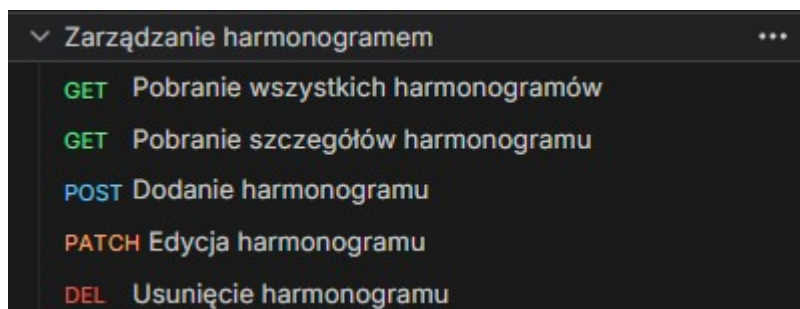
Test walidacji usuwania grupy: Sprawdzono zabezpieczenie bazy danych. Wysłano żądanie HTTP DELETE dla identyfikatora grupy, która posiadała przypisanych członków. Zgodnie z założeniami biznesowymi, system zwrócił błąd walidacji i zablokował transakcję. Następnie przetestowano ścieżkę pozytywną na nowo utworzonej, pustej grupie – usunięcie zakończyło się sukcesem (kod 200 OK).

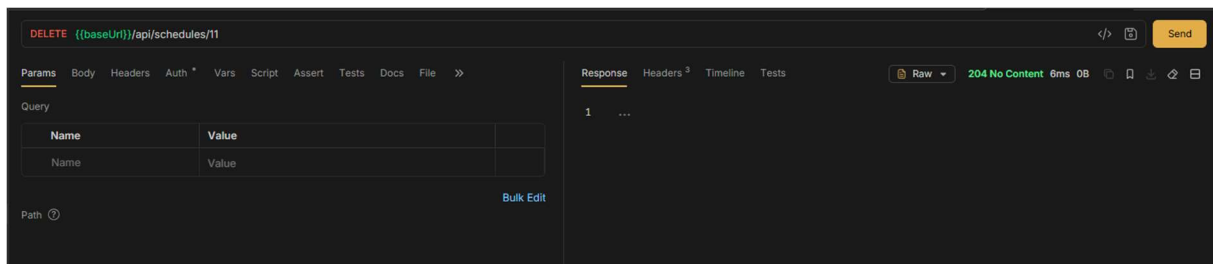
5.6 Testy organizacji wydarzeń i harmonogramów (Event Coordination)

Lp.	Nazwa testu	Metoda	Endpoint	Oczekiwany rezultat
1	Zarejestruj intencję	POST	/api/intentions	System zapisuje intencję mszalną
2	Utwórz harmonogram	POST	/api/schedules	System dodaje nowy slot w harmonogramie
3	Przypisz wydarzenie	POST	/api/events	System tworzy wydarzenie podpięte pod harmonogram
4	Próba usunięcia zajętego harmonogramu	DELETE	/api/schedules/{id}	Operacja usunięcia zostaje zablokowana



Test dotyczył sprawdzenia endpointu rejestrującego intencje parafialne. System poprawnie odczytał przekazane parametry JSON i zapisał je w relacyjnej bazie danych.





Test walidacji użycia harmonogramu: Przeprowadzono test negatywny w celu weryfikacji blokady usuwania zajętych slotów. Wysłano żądanie DELETE na endpoint harmonogramu, do którego uprzednio przypisano wydarzenie parafialne. System, w trosce o integralność zdarzeń, zablokował usunięcie rekordu. Udało się usunąć rekord jedynie na pustym (wolnym) obiekcie harmonogramu.

5.7 Podsumowanie

Przeprowadzone testy potwierdziły poprawne działanie wszystkich najważniejszych funkcji systemu zarządzania parafią. Zweryfikowano procedury ewidencyjne, duszpasterskie i kancelaryjne. Testy w sposób szczególny uwzględniły restrykcyjne zasady walidacji biznesowej (takie jak ograniczenia przy usuwaniu kartotek, grup i harmonogramów).

Wszystkie zależności i wymogi dotyczące kolejności wykonywania operacji zostały poprawnie obsłużone przez aplikację. System właściwie komunikował błędy, uniemożliwiając powstawanie tzw. "osieroconych" rekordów w bazie danych. Na podstawie wykonanych scenariuszy stwierdzono, że aplikacja eParafia działa stabilnie i w pełni realizuje złożone założenia architektoniczne.

6 Informacje dodatkowe

6.1 Role w projekcie

Analitycy:

Anita Gumieniak

Piotr Kut

Marta Flis

Faustyna Gryboś

Projektanci:

Mateusz Domin

Mateusz Draus

Programiści:

Mikołaj Data

Maciej Gilecki

Paweł Górski

Mateusz Gałda

Testerzy:

Aleksandra Gabro

Wioletta Grabias

Jakub Dul